

Documentação Técnica - AdotaPet

Prova de Conceito (PoC) – AEP 2026.2 – Engenharia de Software

1. Apresentação

Este documento descreve a primeira entrega técnica do projeto AdotaPet, uma PoC destinada ao registro e ao gerenciamento de animais para adoção. A documentação apresenta o escopo implementado, a arquitetura e organização da solução, o modelo de dados, os contratos dos endpoints, as regras de validação, a estratégia de testes e os procedimentos necessários para execução.

A primeira entrega concentra-se no núcleo de cadastro e gerenciamento de animais. O sistema permite registrar animais, consultar os registros armazenados, atualizar informações, filtrar registros pelo status de adoção, registrar a adoção e remover animais.

O notar que abrigos e protetores independentes enfrentam dificuldades para centralizar e organizar as informações dos animais resgatados ou abandonados que estão disponíveis para adoção. O objetivo do sistema é disponibilizar uma API para organizar informações de animais destinados à adoção. A aplicação centraliza os dados básicos de cada animal e controla a mudança de status entre disponibilidade e adoção.

O projeto está alinhado ao **Objetivo de Desenvolvimento Sustentável 11 — Cidades e Comunidades Sustentáveis**, pois contribui para uma organização mais eficiente de informações relacionadas ao bem-estar animal e à adoção responsável.

2. Configuração da aplicação

A configuração atual está no arquivo *src/main/resources/application.properties*:

<code>spring.application.name=aep6s</code>
<code>spring.data.mongodb.uri=mongodb://localhost:27017/adotapet</code>
<code>server.port=\${SERVER_PORT:8082}</code>

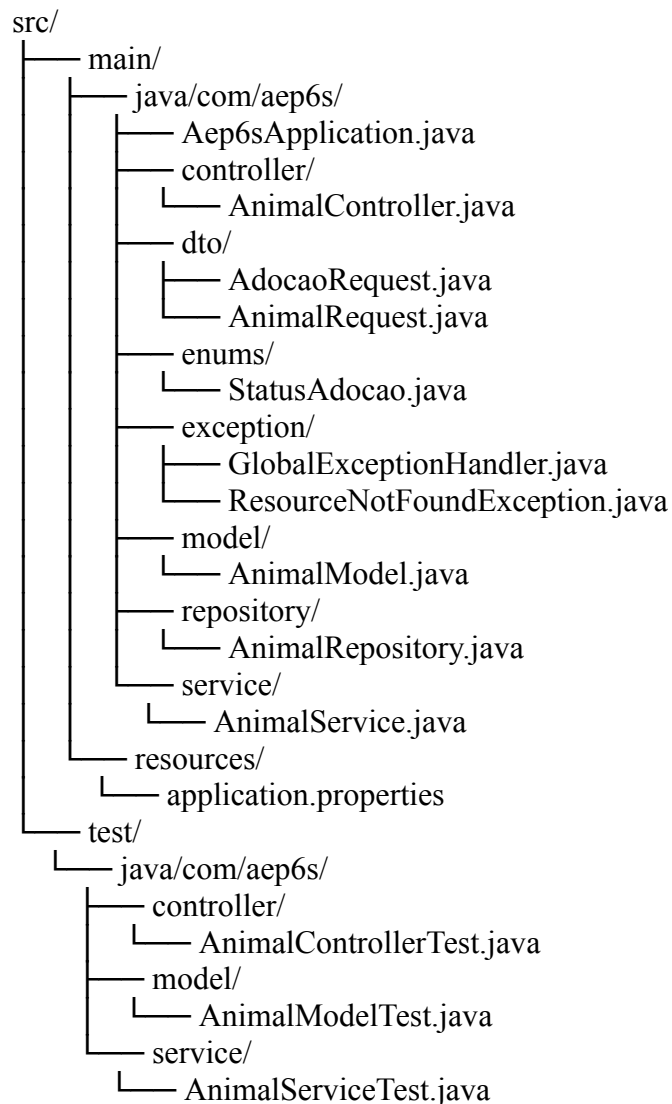
3. Banco de Dados

O projeto utiliza o MongoDB como banco de dados NoSQL. Na primeira entrega, é utilizada a coleção 'animais'. Os registros armazenam informações como nome, espécie, raça, idade, porte, sexo, descrição, status, adotante e data de cadastro.

4. Arquitetura e organização

A aplicação utiliza uma arquitetura em camadas. O controlador recebe as requisições HTTP, o serviço executa as regras de negócio e o repositório realiza a comunicação com o MongoDB.

- **Controller:** recebe e responde às requisições HTTP.
- **Service:** concentra as regras e operações da aplicação.
- **Repository:** realiza o acesso aos dados do MongoDB.
- **Model:** representa os animais armazenados.
- **DTO:** representa os dados recebidos nas requisições.
- **Exception:** realiza o tratamento padronizado de erros.



5.Endpoints

A API disponibiliza os seguintes endpoints:

Método	Rota	Descrição
POST	/animais	Cadastrar um novo animal
GET	/animais	Listar todos os animais
GET	/animais/{id}	Buscar um animal por id
GET	/animais/status/{status}	Listar por status (DISPONIVEL/ADOTADO)
PUT	/animais/{id}	Atualizar os dados de um animal
PUT	/animais/{id}/adotar	Registrar a adoção (nome do adotante)
DELETE	/animais/{id}	Remover um animal

6. Exemplo de validação

Com o MongoDB e a aplicação em execução, utilize o seguinte fluxo no Postman.

6.1 Cadastro

POST http://localhost:8082/animais
<pre>{ "nome": "Thor", "especie": "cachorro", "raca": "SRD", "idade": 3, "porte": "medio", "sexo": "macho", "descricao": "Dócil" }</pre>

6.2 Consulta

GET http://localhost:8082/animais

6.3 Adoção

Substitua *{id}* pelo identificador retornado no cadastro:

PUT http://localhost:8082/animais/{id}/adotar
<pre>{ "adotante": "Maria Silva" }</pre>

7. Validação e tratamento de erros

A API utiliza validações para evitar dados inválidos, como:

- Nome do animal obrigatório;
- Espécie obrigatória;
- Porte obrigatório;
- Sexo obrigatório;
- Idade não pode ser negativa;
- Nome do adotante obrigatório ao registrar uma adoção.

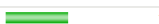
Também existem tratamentos para situações como:

- Animal não encontrado: 404 Not Found;
- Animal já adotado: 409 Conflict;
- Dados inválidos: 400 Bad Request.

8. Cobertura dos testes

Na versão atual da PoC, foram executados um total de 22 testes, 0 falhas, 0 erros e 0 testes ignorados. A cobertura total registrada no relatório JaCoCo é de aproximadamente 94%, ultrapassando o mínimo de 70% exigido para a primeira entrega. Como apresentado na imagem abaixo:

aep6s

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed Cxty	Missed Lines	Missed Methods	Missed Classes
com.aep6s.model		90%		n/a	3 26	6 52	3 26	0 1
com.aep6s.exception		87%		n/a	1 7	1 14	1 7	0 2
com.aep6s		37%		n/a	1 2	2 3	1 2	0 1
com.aep6s.service		100%		100%	0 10	0 33	0 9	0 1
com.aep6s.dto		100%		n/a	0 18	0 26	0 18	0 2
com.aep6s.controller		100%		n/a	0 8	0 12	0 8	0 1
com.aep6s.enums		100%		n/a	0 1	0 3	0 1	0 1
Total	25 of 450	94%	0 of 2	100%	5 72	9 143	5 71	0 9

O relatório também pode ser executado e encontrado após a execução dos testes em <target/site/jacoco/index.html>.